

Calcul scientifique sur Python

Pr A. OUSAADANE

UMI, ENS

23/04/2026



Plan de cours

- 1 Introduction aux modules
- 2 NumPy : Calcul scientifique de base
- 3 Matplotlib : Visualisation
- 4 SciPy : Méthodes numériques avancées
- 5 SymPy : Calcul formel

- 1 Introduction aux modules
- 2 NumPy : Calcul scientifique de base
- 3 Matplotlib : Visualisation
- 4 SciPy : Méthodes numériques avancées
- 5 SymPy : Calcul formel

Qu'est-ce qu'un module ?

Définition

- Un module est un fichier contenant des fonctions, classes et variables réutilisables.
- Python dispose de modules **standards** (math, random, os).
- Des modules **externes** peuvent être installés (NumPy, SciPy, Matplotlib, SymPy).

Installation et importation

Installation avec pip

```
pip install numpy  
pip install numpy scipy matplotlib sympy
```

Importation des modules

```
import numpy as np  
import matplotlib.pyplot as plt  
import scipy as sp  
import sympy as sy
```

À éviter

`from numpy import *` peut causer des conflits de noms.

- 1 Introduction aux modules
- 2 NumPy : Calcul scientifique de base**
- 3 Matplotlib : Visualisation
- 4 SciPy : Méthodes numériques avancées
- 5 SymPy : Calcul formel

Pourquoi NumPy ?

NumPy (Numerical Python)

- Bibliothèque fondamentale pour le calcul scientifique.
- Structure centrale : `ndarray` (tableau multidimensionnel).
- Opérations **vectorisées** (sans boucles explicites).
- Bien plus rapide que les listes Python pour les grands volumes de données.

À partir d'une liste

```
import numpy as np
a = np.array([1, 2, 3])           # vecteur
b = np.array([[1, 2], [3, 4]])   # matrice 2x2
```

Fonctions de création usuelles

```
zeros = np.zeros((2, 3))         # matrice de zeros 2x3
ones = np.ones((2, 2))           # matrice de uns 2x2
identite = np.eye(3)             # matrice identite 3x3
x = np.linspace(0, 1, 10)        # 10 points entre 0 et 1
```


Informations sur un tableau

```
a = np.array([[1, 2, 3], [4, 5, 6]])

print(a.shape)    # (2, 3) : 2 lignes, 3 colonnes
print(a.size)     # 6 : nombre total d'elements
print(a.ndim)     # 2 : nombre de dimensions
print(a.dtype)    # int64 : type des elements
```

Accès aux éléments

```
a = np.array([[1, 2, 3], [4, 5, 6]])

print(a[0, 1])      # 2 (ligne 0, colonne 1)
print(a[:, 1])      # [2, 5] (toutes lignes, colonne 1)
print(a[0, :])      # [1, 2, 3] (ligne 0)
print(a[1, 1:])      # [5, 6] (ligne 1, colonnes 1 à fin)
```

Opérations élément par élément

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)      # [5, 7, 9]
print(a * b)      # [4, 10, 18]
print(a ** 2)     # [1, 4, 9]
print(np.sqrt(a)) # [1, 1.41, 1.73]
```

Broadcasting (diffusion)

```
print(a + 10)     # [11, 12, 13]
```

Statistiques de base

```
a = np.array([1, 2, 3, 4, 5])

print(np.mean(a))    # moyenne : 3.0
print(np.sum(a))     # somme : 15
print(np.std(a))     # ecart-type : ~1.41
print(np.var(a))     # variance : 2.0
print(np.min(a))     # minimum : 1
print(np.max(a))     # maximum : 5
```

Opérations matricielles

```
A = np.array([[1, 2], [3, 4]])  
B = np.array([[5, 6], [7, 8]])  
  
print(A @ B)           # produit matriciel  
print(A.T)             # transposee  
print(np.linalg.det(A)) # determinant  
print(np.linalg.inv(A)) # inverse
```

@ est l'opérateur de multiplication matricielle.

Génération de nombres aléatoires

```
# Nombres aleatoires uniformes dans [0, 1)
rand = np.random.rand(2, 3)    # matrice 2x3

# Entiers aleatoires entre 0 et 10
randint = np.random.randint(0, 10, size=(2, 3))

# Loi normale (moyenne=0, ecart-type=1)
normal = np.random.randn(100)
```

Fixer la graine (reproductibilité)

```
np.random.seed(42)    # resultats reproductibles
```

Reshape et transformation

Changement de forme

```
a = np.array([1, 2, 3, 4, 5, 6])  
b = a.reshape(2, 3)    # matrice 2x3  
print(b)  
# [[1, 2, 3],  
#   [4, 5, 6]]
```

Aplatissement

```
c = b.flatten()          # retour à [1, 2, 3, 4, 5, 6]
```

- 1 Introduction aux modules
- 2 NumPy : Calcul scientifique de base
- 3 Matplotlib : Visualisation**
- 4 SciPy : Méthodes numériques avancées
- 5 SymPy : Calcul formel

Introduction à Matplotlib

Matplotlib

- Bibliothèque de visualisation inspirée de MATLAB.
- Intégration parfaite avec NumPy.
- Module principal : `pyplot`.

Importation standard

```
import matplotlib.pyplot as plt  
import numpy as np
```

Tracé de la fonction $\sin(x)$

```
x = np.linspace(0, 2*np.pi, 100)
y = np.sin(x)

plt.plot(x, y)
plt.show()
```

`linspace` crée 100 points équitablement espacés.

Titre, axes, grille

```
plt.plot(x, y, color='b', linestyle='-', linewidth=2)
plt.title("Fonction sinus")
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.grid(True)
plt.show()
```

Couleurs et styles courants

- Couleurs : 'r' (rouge), 'g' (vert), 'b' (bleu), 'k' (noir)
- Styles : '-' (trait), '-' (tirets), ':' (pointillés)
- Marqueurs : 'o' (cercle), 's' (carré), '*' (étoile)

Superposition de $\sin(x)$ et $\cos(x)$

```
y1 = np.sin(x)
y2 = np.cos(x)

plt.plot(x, y1, 'b-', label='sin(x)')
plt.plot(x, y2, 'r--', label='cos(x)')
plt.legend()
plt.title("Sinus et cosinus")
plt.show()
```

Figure avec 2 sous-graphiques côte à côte

```
fig, axs = plt.subplots(1, 2, figsize=(10, 4))

axs[0].plot(x, np.sin(x))
axs[0].set_title("sin(x)")

axs[1].plot(x, np.cos(x))
axs[1].set_title("cos(x)")

plt.show()
```

Autres types de graphiques

Nuage de points (scatter)

```
x = np.random.randn(100)
y = 2*x + np.random.randn(100)
plt.scatter(x, y)
```

Histogramme

```
data = np.random.randn(1000)
plt.hist(data, bins=30)
```

Diagramme en barres

```
categories = ['A', 'B', 'C']
valeurs = [10, 15, 7]
plt.bar(categories, valeurs)
```

- 1 Introduction aux modules
- 2 NumPy : Calcul scientifique de base
- 3 Matplotlib : Visualisation
- 4 SciPy : Méthodes numériques avancées**
- 5 SymPy : Calcul formel

SciPy (Scientific Python)

- Bibliothèque construite sur NumPy.
- Sous-modules pour des problèmes scientifiques courants.

Sous-modules importants

- `scipy.optimize` : minimisation, racines d'équations
- `scipy.integrate` : intégration numérique, EDO
- `scipy.linalg` : algèbre linéaire avancée
- `scipy.stats` : statistiques

Recherche du minimum d'une fonction

```
from scipy.optimize import minimize

def f(x):
    return x**2 + 3*x + 2

result = minimize(f, x0=0)
print(f"Minimum en x = {result.x[0]:.4f}")
print(f"f(min) = {result.fun:.4f}")
```

Pour $f(x) = x^2 + 3x + 2$, le minimum est en $x = -1.5$.

Optimisation : racines d'une équation

Résolution de $f(x) = 0$ avec `fsolve`

```
from scipy.optimize import fsolve

def f(x):
    return x**3 - 2*x - 5

racine = fsolve(f, x0=2)
print(f"Racine : {racine[0]:.6f}")
```

Utile pour des équations sans solution analytique.

Résolution de $\frac{dy}{dt} = -2y$ avec $y(0) = 1$

```
from scipy.integrate import solve_ivp
```

```
def edo(t, y):  
    return -2 * y
```

```
solution = solve_ivp(edo, [0, 5], [1])  
t = solution.t  
y = solution.y[0]
```

```
plt.plot(t, y)  
plt.title("Solution de  $dy/dt = -2y$ ")  
plt.show()
```

Décomposition LU

```
from scipy.linalg import lu

A = np.array([[3, 2], [1, 4]])
P, L, U = lu(A)

print("L :\n", L)
print("U :\n", U)
```

Résolution de système linéaire

```
from scipy.linalg import solve

A = np.array([[3, 2], [1, 4]])
b = np.array([1, 2])
x = solve(A, b)
print(x)
```

- 1 Introduction aux modules
- 2 NumPy : Calcul scientifique de base
- 3 Matplotlib : Visualisation
- 4 SciPy : Méthodes numériques avancées
- 5 SymPy : Calcul formel**

SymPy (Symbolic Python)

- Bibliothèque de **calcul formel** (Computer Algebra System).
- Résultats **exacts** (fractions, racines symboliques).
- Alternative libre à Mathematica ou Maple.

Importation

```
import sympy as sy
```

Déclaration de symboles

```
x, y, z = sy.symbols('x y z')  
f = x**2 + 2*x + 1  
print(f)
```

Évaluation numérique

```
print(f.subs(x, 2))           # 9  
print(f.subs(x, 2).evalf())   # 9.0
```


Développer une expression

```
expr = (x + 1)**2 * (x - 1)
developpe = sy.expand(expr)
print(developpe)
```

Factoriser

```
factorise = sy.factor(developpe)
print(factorise)
```

Simplification

```
expr = sy.sin(x)**2 + sy.cos(x)**2
print(sy.simplify(expr))
```

Dérivée première et seconde

```
f = x**3 + 2*x**2 + x
f_prime = sy.diff(f, x)
print(f_prime)

f_second = sy.diff(f, x, 2)
print(f_second)
```

Dérivée partielle

```
f = x**2 * y + y**3
df_dx = sy.diff(f, x)
df_dy = sy.diff(f, y)
```

Intégrale définie et indéfinie

```
f = x**2

# Intégrale indéfinie
F = sy.integrate(f, x)
print(F)

# Intégrale définie de 0 à 1
I = sy.integrate(f, (x, 0, 1))
print(I)
```

Intégrale double

```
f = x*y
I2 = sy.integrate(f, (x, 0, 1), (y, 0, 1))
print(I2)
```

Équation polynomiale

```
equation = x**2 - 4
solutions = sy.solve(equation, x)
print(solutions)
```

Système d'équations

```
eq1 = x + y - 5
eq2 = x - y - 1
solutions = sy.solve((eq1, eq2), (x, y))
print(solutions)
```

Résolution d'équations différentielles

EDO simple : $y'' + y = 0$

```
y = sy.Function('y')
edo = sy.diff(y(x), x, 2) + y(x)
solution = sy.dsolve(edo, y(x))
print(solution)
```

Avec conditions initiales

```
solution = sy.dsolve(edo, y(x),
ics={y(0): 1, sy.diff(y(x), x).subs(x, 0): 0})
print(solution)
```

Création et opérations

```
A = sy.Matrix([[1, 2], [3, 4]])  
B = sy.Matrix([[5, 6], [7, 8]])  
  
print(A + B)  
print(A * B)  
print(A.det())  
print(A.inv())
```

Matrice avec paramètre symbolique

```
a = sy.symbols('a')  
M = sy.Matrix([[a, 1], [1, a]])  
print(M.det())
```

Bonnes pratiques

- Utiliser des alias standard : `np`, `plt`, `sp`, `sy`.
- Préférer les opérations vectorisées aux boucles explicites.

Acquis

- Installer et importer des modules scientifiques.
- Manipuler des tableaux NumPy.
- Visualiser des données avec Matplotlib.
- Utiliser SciPy pour l'optimisation, les EDO et l'algèbre linéaire.
- Effectuer du calcul formel exact avec SymPy.

Prochain chapitre : Analyse de données avec Pandas